System overview

What runs where, where each kind of data lives, and every connection between the parts. Numbered arrows are explained at the bottom of the page.

1 sign in / sign up / reset → JWT access + refresh token

BROWSER — React 19 SPA · skimmi/frontend/src

AuthGate.jsx · AuthScreen.jsx

Sign in, sign up, forgot and reset password (Supabase Auth via **supabase-js**, public anon key). Renders the app only once a session exists.

- session kept in localStorage and refreshed automatically

App.jsx — all application state

- documents (re-polled every 4 s while one is `processing`)
- chats; the open chat id lives in the URL `#chat=<id>`
- study **mode** and **scope** (document or subject, pages, topic)
- progress, uploads, toasts; a `view` counter drops replies for a chat the user left

UI components

- **Sidebar**: upload, documents by subject, Progress card, chats, account
- **Composer** with ModeMenu + ScopeMenu chips
- **Messages / Study.jsx**: markdown + KaTeX answers, one-question-at-a-time quiz, flashcards
- **Sources**: citation pill → passage excerpts
- **AccountPanel**: account counts, change password, sign out

api.js — the only door to the backend

- sends `Authorization: Bearer <access token>` on every `/api` call
- 401 → refresh the session once and retry, else sign out
- uploads use XHR so the upload % can be shown
- shows the server's `{detail}` message on errors

Vite dev server

`:5173` serves the SPA and proxies `/api` → `:8000`

HTTPS /api/* + Bearer JWT

2

BACKEND — FastAPI + Uvicorn :8000 · skimmi/backend/app · one process

main.py — middleware & error handlers

CORS allow-list · body limit (25 MB upload, 256 KB JSON → 413) · security headers · request log · errors are always `{detail, code}`; outages → 503

ratelimit.py

Sliding windows in memory: chat 15/min · upload 30/h · write 120/min · bad tokens 30 per 5 min per IP → 429

auth.py — current_user

Verifies the Supabase JWT (signature, exp, aud, iss, role). User id = token `sub`, never the request.

public keys (JWKS), cached 10 min — 3

main.py — REST API (all need a user except /health)

`GET /api/health` · `GET /api/me` · `GET /api/progress`
`GET POST /api/documents` · `PATCH DELETE /api/documents/{id}`
`POST /api/documents/{id}/reindex` · `GET …/{id}/file`
`POST /api/chat` · `GET DELETE /api/conversations[/{id}]`
`POST /api/messages/{id}/quiz-attempts`

rag.py — the one RAG pipeline

extract → `split_text` → `index_chunks` → `retrieve_context` → `call_gemini`

- `run_mode`: ask · explain · summarize · quiz · flashcards
- grounded prompts, validated quiz/flashcard JSON, model fallbacks

8 prompt → answer / JSON

7

Embedding model (in memory)

`all-MiniLM-L6-v2` · normalized vectors · loaded once

extract.py

PDF, EPUB, DOCX, PPTX, TXT, MD → pages of text

vectors.py

The only ChromaDB code: add · query · get · count · delete. Every filter includes `user_id`.

Startup: migrate DB → ensure bucket → load model + Chroma → background recover_index

DATA ADAPTERS

progress.py

Scores quizzes on the server; progress summary.

database.py

SQLAlchemy Core + Alembic. Every function takes `user_id` and filters on it: another user's id → 404.

SQL — 4

storage.py

Original files at `<user>/<doc>.<ext>` via Storage REST + service-role key (local folder in tests).

REST — 5

6

BACKEND HOST DISK — skimmi/backend/data/ (gitignored)

uploads/

file store for tests / offline dev

skimmi-dev.db

SQLite used when DATABASE_URL is unset

ChromaDB chroma_db/

collection `skimmi_documents` — can be rebuilt from Storage

SUPABASE — managed cloud project

Auth

Email + password accounts, confirmation and reset emails. Issues short-lived JWT access tokens and publishes its public signing keys (JWKS).

PostgreSQL

`profiles` · `documents` · `conversations` · `messages` · `quiz_attempts` — every row has a `user_id`

- schema created by Alembic migration 0001
- Row Level Security: users may SELECT only their own rows; no client writes
- `profiles.id` → `auth.users` ON DELETE CASCADE

Storage — private bucket "documents"

The original uploaded files at `<user_id>/<doc_id>.<ext>`. No public URLs: only the backend can read them (service-role key), after checking the document belongs to the user.

GOOGLE AI

Gemini API (google-genai)

- main model `gemini-3.8-flash`
- on overload or quota: 3.5-flash → 3.6-flash → 3.5-flash-lite → flash-lite-latest
- 60 s timeout; JSON output for quizzes and flashcards
- receives only the question, the retrieved passages and the last 6 chat messages

HOW TO READ THIS PAGE

- Browser (runs on the student's device)
- Backend code (Python, one process)
- Supabase managed services
- Backend host: disk and memory
- Google Gemini (external AI)
- Dev tooling

↔ request + response

→ one-way call

→ internal call

NUMBERED CONNECTIONS

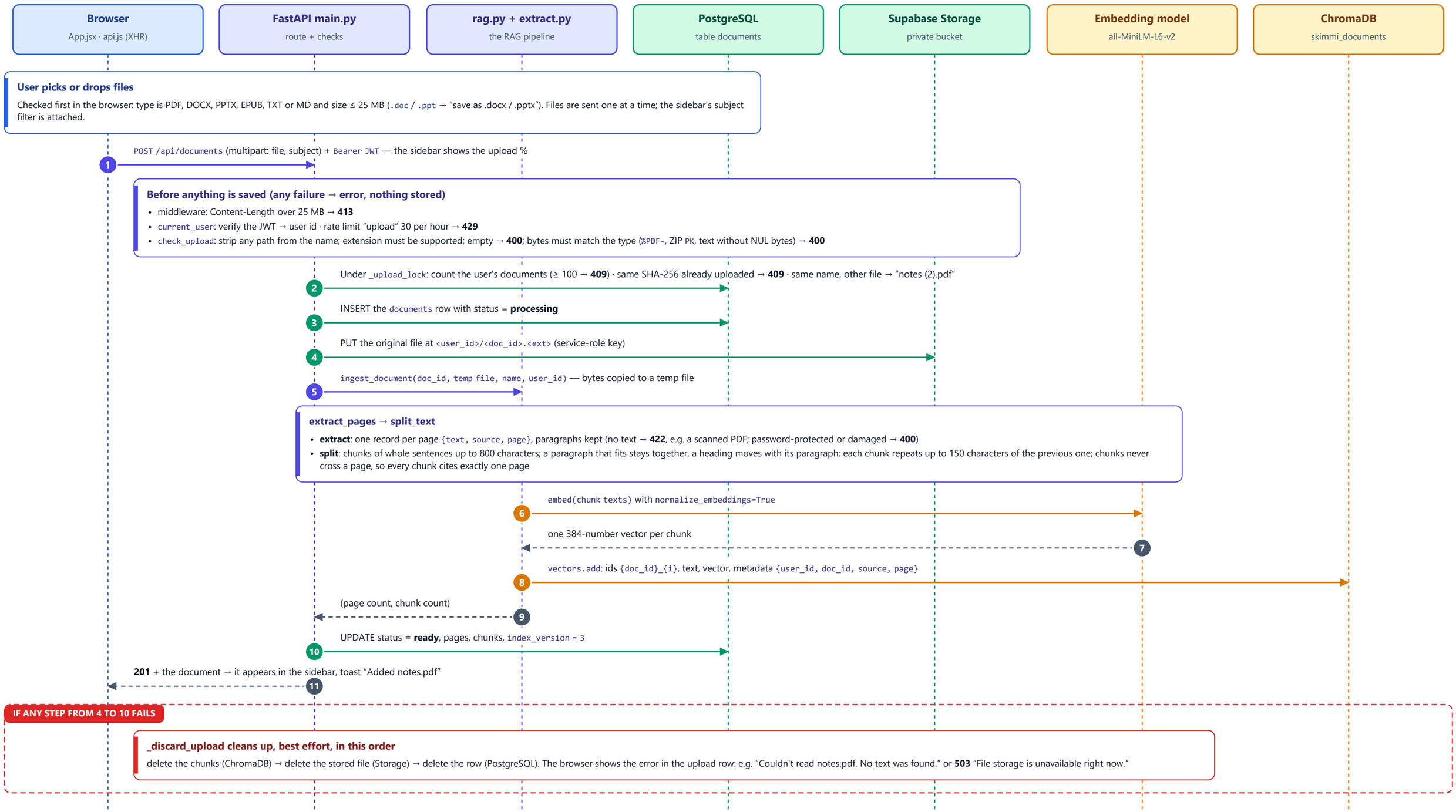
1. **Sign-in.** supabase-js talks to Supabase Auth directly and receives a JWT access token (refreshed automatically). The backend never sees passwords.
2. **API calls.** Every `/api/*` request goes through the Vite proxy to FastAPI with the token as a Bearer header.
3. **Token check.** auth.py verifies the token with the project's public keys and takes the user id from it.
4. **Metadata.** database.py reads and writes rows (documents, chats, quiz results), always filtered by the user id.
5. **Files.** storage.py saves and reads the original uploads in the private bucket.
6. **Chunks.** vectors.py adds, searches and deletes text chunks + embeddings, always limited to the user's chunks.
7. **Embeddings.** rag.py turns chunks and search queries into vectors with the local model.
8. **Answers.** rag.py sends a grounded prompt to Gemini and gets back an answer, summary, or quiz / flashcard JSON.

RULES THAT HOLD EVERYWHERE

- one RAG pipeline (rag.py), one vector collection, one file store
- every row, file path and chunk carries the owner's `user_id`
- ChromaDB persists on disk; at startup only documents with missing or outdated chunks are re-indexed from Storage
- a question and its answer are saved only when the answer succeeds

Uploading and indexing a document

What happens, in order, from dropping a file in the browser to it being searchable. Time runs downward; each numbered arrow is one call between the parts.



AFTER A SUCCESSFUL UPLOAD THE DOCUMENT EXISTS IN THREE PLACES, ALL TAGGED WITH THE OWNER

PostgreSQL · documents row

name, SHA-256, size, subject, pages, chunk count, status = ready, index_version = 3. This is what the sidebar lists.

Storage · the original file

`<user_id>/<doc_id>.pdf`, byte for byte. Used for downloads and to rebuild the index; never changed after upload.

ChromaDB · the chunks

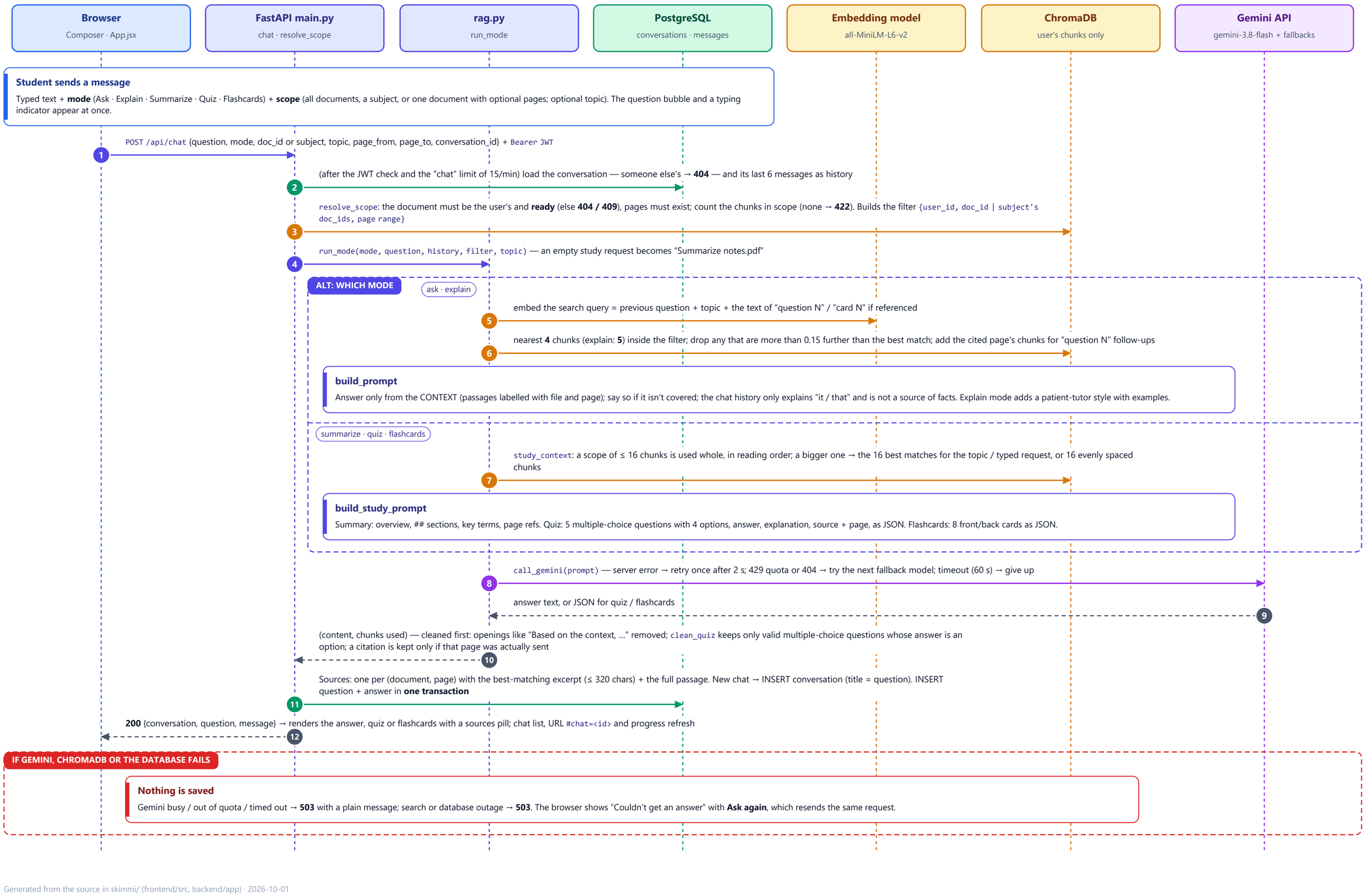
`{doc_id}_0 … {doc_id}_n`, each with text, vector and `{user_id, doc_id, source, page}`. What questions are answered from.

Same file again?

Same bytes → refused (409). Different file, same name → kept as "notes (2).pdf". Two users may upload the same file: everything is per user.

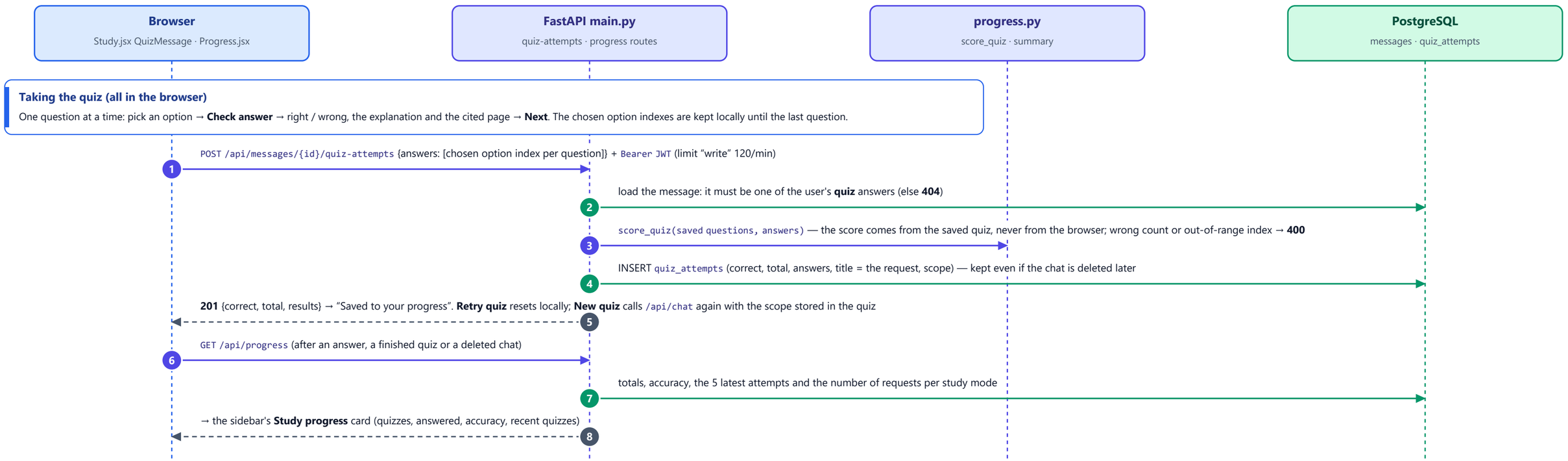
Asking a question or using a study mode

One request to POST /api/chat, from the composer to the saved answer. The same route serves all five modes; they differ only in how material is gathered and what Gemini is asked to write.

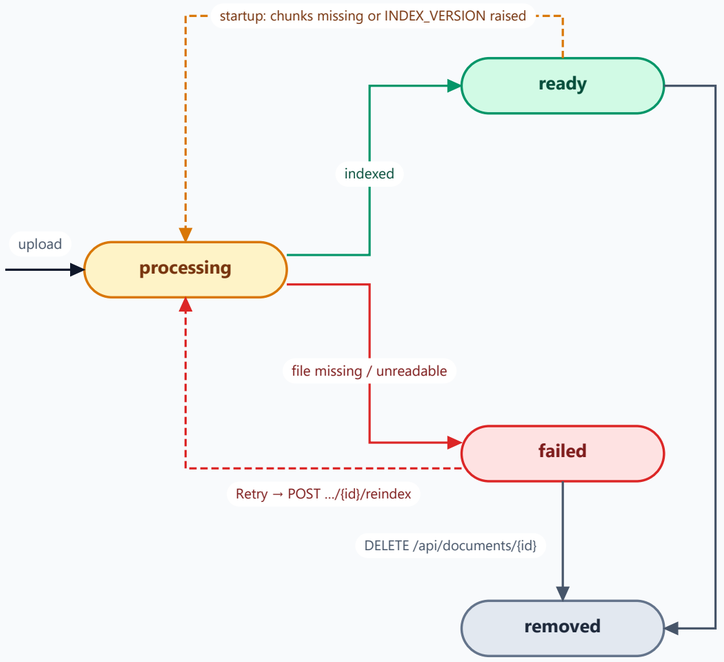


Quizzes, progress and the document lifecycle

Top: how a finished quiz is scored and turned into the progress card. Bottom: the states a document goes through, how the search index heals itself, and what a cited “page” means for each file type.



DOCUMENT STATUS (documents.status)



Only **ready** documents can be chosen as study material. While any document is processing, the browser re-reads the list every 4 s.

INDEX RECOVERY (ChromaDB is a rebuildable cache)

Backend starts
`recover_index` runs in a background thread; requests are served meanwhile

For each document that isn't failed (all users)
needs work if status ≠ ready, `index_version` < 3, or it has no chunks in ChromaDB

→ **Nothing to do**: skip it

needs work

reindex_from_storage
download the file from Storage → extract → split → embed, **then** swap the old chunks for the new ones and mark it **ready** (a failure leaves the old chunks working)

→ **File missing / unreadable** → **failed**

→ **Storage / Chroma down** → try again next start

Then remove orphaned chunks
chunks whose document no longer exists (e.g. the account was deleted in Supabase)

The same `reindex_from_storage` runs when the user presses **Retry** on a failed document. To re-index everything after changing chunking or the embedding model: raise `INDEX_VERSION` in config.py and restart.

WHAT A CITED “PAGE” IS

PDF, EPUB	the real pages (PyMuPDF text blocks → paragraphs)
PowerPoint	one page per slide, plus its speaker notes
Word	Word's saved page breaks, else ~3,000-character sections
Text, Markdown	~3,000-character sections (Markdown syntax removed)
Tables	one row per paragraph, cells joined by “ | ”

No OCR: a scanned, image-only PDF has no text and is rejected.

DELETING A DOCUMENT — ORDER MATTERS

1. delete chunks (ChromaDB) → **2. delete file** (Storage) → **3. delete row** (PostgreSQL)

If any step fails the document is still listed, so pressing delete again finishes the job. Nothing is ever left that the user can't see.

ONE CHUNK AS STORED IN CHROMADB

id 3f2a9c1e-…-b7d4_12

`document` “1.3 Integrity ¶ Integrity means that data is not changed by anyone who isn't allowed to…” (≤ 800 chars)
`embedding` [0.021, -0.044, 0.113, … 384 numbers, length 1]
`metadata` {user_id: “11111111-…”, doc_id: “3f2a9c1e-…”, source: “notes.pdf”, page: 4}

Security model and data model

Left: the checks every request passes before it can touch data. Right: what is stored, where, and how the records point at each other. Everything belongs to exactly one user.

A REQUEST'S PATH THROUGH THE SECURITY LAYERS

1. **Browser holds only public keys**
   supabase-js keeps the session; the frontend has the public anon key. The service-role key and the Gemini key live only in the backend's .env.

2. **Middleware (main.py)**
   Only the configured origins may call the API (never "*"). Bodies over the limit are refused before they are read. Adds nosniff, DENY framing, no-referrer, no-store. — 413

3. **Who is this? (auth.py)**
   A Bearer JWT is required. Signature checked with the project's public keys (or the legacy HS256 secret); exp, aud = authenticated, iss = this project, role = authenticated, sub is a UUID. — 401 · 503

4. **Slow down abuse (ratelimit.py)**
   30 bad tokens per 5 min per IP; per user: chat 15/min, upload 30/h, other writes 120/min. In memory, per process. — 429

5. **Validate the input**
   Pydantic models (lengths, mode names, page numbers); uploads: type, size and the file's real bytes. — 400 · 422

6. **Ownership in every query (database.py)**
   Every read and write filters on the user id from the token. Another user's id, or a malformed one, gives the same 404 as an id that doesn't exist. — 404

7. **Search isolation (rag.py)**
   `scope_filter` always adds `user_id`; `_check_scope` refuses any search without it, so retrieval can never reach another user's chunks.

8. **File isolation (storage.py)**
   Paths are built only from UUIDs: `<user_id>/<doc_id>.<ext>`. The bucket is private; files are served only through `/api/documents/{id}/file` after the ownership check.

9. **Row Level Security (defence in depth)**
   If someone calls Supabase's Data API directly with the anon key: users can SELECT only their own rows and cannot write at all. The backend connects as the owner and bypasses RLS.

10. **Safe errors**
    Every error is `{detail, code}`. Outages → 503 with a plain message; anything else → a generic 500. No stack traces, config hints or tokens in responses or logs. — 503 · 500

Numbers 6–9 are why an id from someone else's account behaves exactly like an id that doesn't exist, everywhere: database rows, search results and files.

Where the secrets live

- **backend/.env** (server only): `GEMINI_API_KEY`, `SUPABASE_SERVICE_ROLE_KEY` (bypasses RLS, used only for Storage), `DATABASE_URL` (Postgres password), optional `SUPABASE_JWT_SECRET`
- **frontend/.env** (public, shipped to browsers): `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` — safe because of layers 6–9
- both .env files are gitignored; only the .env.example templates are committed

SUPABASE POSTGRESQL

5 tables · Alembic migration 0001 · RLS on

auth.users

PK	id	uuid
	email	text

managed by Supabase Auth

id · ON DELETE CASCADE

profiles

PK	id	uuid
	email	varchar(320)
	created_at	timestamptz
	last_seen_at	timestamptz

created on the user's first request

1 → many · CASCADE

conversations

PK	id	uuid
FK	user_id	uuid
	title	varchar(200)
	created_at	timestamptz
	updated_at	timestamptz

documents

PK	id	uuid
FK	user_id	uuid
	filename	varchar(255)
	sha256	varchar(64)
	extension	varchar(10)
	size_bytes	bigint
	storage_path	unique
	pages	int
	chunks	int
	subject	varchar(60)
	course	varchar(60)
	status	processing|ready|failed
	index_version	int
	error	varchar(300)
	uploaded_at	timestamptz
	updated_at	timestamptz

unique (user_id, sha256) · unique (user_id, lower(filename))

messages

PK	id	bigint
FK	conversation_id	uuid
FK	user_id	uuid
	role	user|assistant
	content	text
	sources	jsonb
	mode	varchar(12)
	scope	varchar(300)
	created_at	timestamptz

content: markdown, or JSON for quizzes and flashcards · deleting the chat deletes its messages

quiz_attempts

PK	id	bigint
FK	user_id	uuid
FK	message_id	bigint
	title	varchar(300)
	scope	varchar(300)
	correct	int
	total	int
	answers	jsonb
	completed_at	timestamptz

message_id → messages.id ON DELETE SET NULL: the score is kept when the chat is deleted

Every table's `user_id` → `profiles.id` ON DELETE CASCADE, so deleting a user in Supabase Auth deletes all of their rows. Only some of these links are drawn.

SUPABASE STORAGE

bucket "documents"

documents/<user_id>/<doc_id>.<ext>

Path = `documents.storage_path`. The original file, byte for byte. Private: no public URL. Read only by the backend for downloads and re-indexing.

CHROMADB

collection skimmi_documents

chunk id = <doc_id>_<n>

`doc_id` = `documents.id`, `user_id` = the owner.
`document` (chunk text ≤ 800 chars) · `embedding` (384 numbers) · `metadata` {user_id, doc_id, source, page}. A cache: rebuilt from Storage when missing or outdated.

KEY SETTINGS (backend/app/config.py, env overrides in backend/.env)

Chunks	800 chars, 150 overlap, never across a page	**Index version**	3 — raise it to re-index every document
Retrieval	top 4 (explain 5), drop > best + 0.15 distance	**Uploads**	25 MB per file, 100 documents per user
Study modes	≤ 16 chunks; quiz 5 (max 10); 8 flashcards (max 15)	**Timeouts**	Gemini 60 s · Supabase 30 s · SQL 15 s
History	last 6 messages, each trimmed to 1,200 chars	**Text pages**	~3,000 chars for TXT / MD / Word without breaks